

FICHE D'ACTIVITÉ TYPE 1

Intitulé de l'activité type : Développer et concevoir des interfaces utilisateur sécurisées. **Contexte de réalisation :** Stage de Développeur Fullstack chez OPIUM DIGITAL (Paris) et projet individuel de certification (Mini CRM SaaS).

Description de la réalisation : Dans le cadre du développement du Mini CRM, j'ai pris en charge la conception et l'implémentation complète des interfaces graphiques (Frontend). L'objectif était de fournir aux utilisateurs indépendants une application web ergonomique (Single Page Application) et totalement sécurisée.

J'ai développé le Frontend en utilisant React.js, associé à l'outil de build Vite. Pour assurer une interface responsive (adaptable sur PC, tablette et mobile), j'ai structuré le design via des approches CSS modernes, en garantissant un haut niveau de fluidité. Les parcours utilisateurs (User Stories) implémentés incluent : la création de comptes, la gestion d'un portefeuille de clients, et un tableau de bord dynamique affichant les devis et factures.

Sur le volet Sécurité et Qualité, j'ai mis un point d'honneur à protéger l'interface contre les vulnérabilités classiques du web (OWASP). L'authentification est gérée via un système de JSON Web Tokens (JWT) générés par le serveur et stockés de façon sécurisée. Côté client (React), j'utilise Axios avec un système d'intercepteurs pour attacher ce token à chaque requête HTTP, assurant que seuls les utilisateurs légitimes accèdent aux routes de l'application. J'ai également anticipé les risques de failles XSS (Cross-Site Scripting) en utilisant les mécanismes de protection natifs de React qui échappent automatiquement les entrées utilisateur lors du rendu du DOM.

FICHE D'ACTIVITÉ TYPE 2

Intitulé de l'activité type : Concevoir et développer la persistance des données et les composants métiers. **Contexte de réalisation :** Projet de certification Mini CRM (SaaS B2B).

Description de la réalisation : Pour répondre aux besoins métiers du CRM, j'ai conçu toute l'architecture Backend et la gestion des données.

1. Modélisation et Persistance des données : J'ai d'abord modélisé l'application en concevant un Modèle Conceptuel des Données (MCD) regroupant les entités User, Client, Devis et Facture. Ce modèle a été traduit en Modèle Physique implémenté sur un système de gestion de base de données relationnelle performant : PostgreSQL. Pour interagir avec la base, j'ai intégré l'ORM Prisma dans mon code Node.js. Prisma m'a permis d'écrire des requêtes typées, sécurisées contre les injections SQL, et de gérer l'évolution de mon schéma de base de données à travers un système robuste de migrations automatisées.

2. Développement des composants métiers (API REST) : J'ai conçu et développé le serveur Backend en Node.js avec le framework Express. J'ai exposé une API RESTful complète permettant la manipulation des ressources (CRUD). J'ai développé des algorithmes métiers complexes, par exemple : la logique de transformation d'un Devis en Facture. Lors de cette opération, mon API vérifie l'état d'acceptation du devis, duplique les lignes de prestations, génère un numéro de facture incrémental unique (ex: FACT-2026-001) et sauvegarde le tout de manière transactionnelle. Enfin, j'ai implémenté un système de génération de fichiers PDF côté serveur à l'aide de Puppeteer, permettant à l'utilisateur de télécharger un export de ses factures mis en forme au format A4, de manière dynamique.

FICHE D'ACTIVITÉ TYPE 3

Intitulé de l'activité type : Organiser le travail et préparer le déploiement. **Contexte de réalisation :** Projet de certification Mini CRM, hébergé sur environnement cloud (OVH).

Description de la réalisation : Mon travail de développement a été encadré par une méthodologie Agile (Kanban). J'ai suivi et documenté mon avancement au travers de User Stories dans un outil de gestion de tickets, tout en garantissant une traçabilité de mon code grâce à des commits réguliers sur GitHub (Versionning Git).

Concernant le déploiement en production, j'ai mis en place une véritable architecture DevOps cloud. L'application complète a été provisionnée sur un serveur distant (VPS chez OVH) sous un nom de domaine dédié (`m-atici.fr`). Pour garantir l'isolation et la scalabilité des composants, j'ai conteneurisé l'application via Docker. Un fichier `docker-compose.yml` orchestre simultanément mes conteneurs : Backend (Node), Frontend (React), Base de données (PostgreSQL) et Cache (Redis).

Pour sécuriser l'accès public, j'ai déployé "Caddy" en tant que reverse-proxy. Caddy s'occupe de router les flux réseau vers les bons conteneurs (Frontend ou API) et génère automatiquement les certificats TLS (Let's Encrypt), forçant la connexion de mes clients en HTTPS. Enfin, pour faciliter mes mises en production futures, j'ai rédigé un script Shell (`deploy-staging.sh`) qui automatise la récupération du code, la reconstruction des images Docker et l'exécution des migrations de base de données, m'assurant des livraisons rapides et sans friction.